

Python
for DA

Summary Session 6



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос



Что такое Polars?



Polars

Это современная библиотека DataFrame, предназначенная для эффективного манипулирования данными и их анализа. В основе - структура данных DataFrame, аналогичную Pandas DataFrame, но с оптимизацией по скорости, использованию памяти и масштабируемости.

Ключевые особенности Polars



DataFrame



Эффективный механизм запросов



Функциональный API



Интеграция с Arrow

Мотивация использования Polars

-  Ограничения производительности
-  Нагрузка на память
-  Проблемы масштабируемости

Polars решает эти проблемы за счет использования характеристик производительности Rust, включая безопасность памяти, модель параллелизма и эффективную обработку параллелизма

Установка необходимых библиотек

```
pip install polars
```

```
pip install pyarrow
```

```
import polars as pl
```



Табличная структура

DataFrame в Polars состоит из строк и столбцов, где каждый столбец представляет собой серию данных одного типа, а каждая строка - наблюдение или запись.



Маркированные столбцы

Столбцы в DataFrame маркированы, что позволяет легко получить доступ к данным с помощью меток столбцов.



Колоночное хранение данных

Внутренние данные в DataFrame хранятся в формате столбцов, что обеспечивает эффективный доступ к данным, манипулирование ими и вычисления.



Неизменяемые данные

DataFrame в Polars неизменяемы, то есть операции обычно создают новые экземпляры DataFrame, а не изменяют существующие.

Экспресс-опрос

?

Какие преимущества у Polars перед Pandas?

Производительность



Polars

Создан с учетом производительности, используя эффективность Rust для обеспечения более высокой скорости выполнения и лучшего использования памяти, особенно для больших наборов данных

Pandas

Реализован на Python и может страдать от ограничений производительности, особенно при работе с большими наборами данных.

Эффективность использования памяти



Polars

Разработан для эффективного использования памяти, что делает его подходящим для работы с большими наборами данных с минимальными затратами памяти.

Pandas

Может потреблять больше памяти, особенно при выполнении операций, связанных с копированием или изменением формы данных.

Функциональный API



Polars

Предлагает функциональный API программирования, позволяющий пользователям выражать операции манипулирования данными с помощью краткого и выразительного синтаксиса

Pandas

API Polars может предложить более чистый и предсказуемый подход для решения определенных задач.

Интеграция с Arrow



Polars

Интегрируется с Apache Arrow, обеспечивая эффективный обмен данными с другими Arrow-совместимыми библиотеками и инструментами.

Pandas

Не имеет встроенной интеграции с Arrow, хотя совместимость может быть достигнута с помощью функций преобразования.

Параллельная обработка



Polars

Оптимизирован для параллельной обработки и может использовать многоядерные процессоры для повышения производительности.

Pandas

В некоторой степени поддерживает многопоточность.

Из CSV

`pl.read_csv(path: str) -> pl.DataFrame`: Считывает данные из CSV-файла в Polars DataFrame.

Параметры:

- `path (str)`: Путь к CSV-файлу.

```
import polars as pl
df = pl.read_csv('data.csv')
```

Из Pandas DataFrame

`pl.from_pandas(df: pd.DataFrame) -> pl.DataFrame`: Преобразует Pandas DataFrame в Polars DataFrame.

Параметры:

- `df (pd.DataFrame)`: Объект Pandas DataFrame.

```
import polars as pl
import pandas as pd
pandas_df = pd.read_csv('data.csv')
df = pl.from_pandas(pandas_df)
```

Из JSON

`pl.read_json(path: str) -> pl.DataFrame`: Считывает данные из JSON-файла в Polars DataFrame.

Параметры: `path (str)`:

- Путь к файлу JSON.

```
import polars as pl
df = pl.read_json('data.json')
```

Из словаря

```
pl.DataFrame(data: Dict[str, List[Any]]) -> pl.DataFrame
```

Пример

```
import polars as pl
data = {
    'A': [1, 2, 3],
    'B': ['foo', 'bar', 'baz'],
    'C': [True, False, True]
}
df = pl.DataFrame(data)
```


Из списка кортежей

```
pl.DataFrame(data: List[Tuple[Any]]) -> pl.DataFrame
```

Пример

```
import polars as pl
data = [
    (1, 'foo', True),
    (2, 'bar', False),
    (3, 'baz', True)
]
df = pl.DataFrame(data, columns=['A', 'B', 'C'])
```

Использование DataFrameBuilder

```
pl.DataFrameBuilder()
builder.add_column(name: str, values: List[Any]) ->
pl.DataFrameBuilder
builder.finish() -> pl.DataFrame
```

Пример

```
import polars as pl
builder = pl.DataFrameBuilder()
builder.add_column('A', [1, 2, 3])
builder.add_column('B', ['foo', 'bar', 'baz'])
builder.add_column('C', [True, False, True])
df = builder.finish()
```

Фреймы Polars

```
import numpy as np
import polars as pl
import numpy as np

strings_data = ["apple", "banana", "cherry", "watermelon", "elderberry"]
integers_data = np.random.randint(1, 100, size=5)
floats_data = np.random.rand(5)

df = pl.DataFrame({
    "fruit": strings_data,
    "quantity": integers_data,
    "price": floats_data
})

print(df.head(3))
print(df.dtypes)
print(df.schema)
print(df.describe())
```

Экспресс-опрос

?

Что делает `print(df.schema)`?



`print(df.schema)`

Выводит схему DataFrame `df`, которая включает информацию о типах данных и именах каждого столбца.

Экспресс-опрос

?

Что делает `df.describe()`?



`df.describe()`

Показывает сводную статистику для каждого столбца в DataFrame.

Характеристики



count - количество наблюдений или строк в наборе данных.



null_count - количество отсутствующих значений в столбце.



mean - среднее арифметическое значение столбца.



std - стандартное отклонение столбца.



min - минимальное значение столбца.



max - максимальное значение столбца.



median - медианное значение, или пятидесятый процентиль, столбца.



25% - двадцать пятый процентиль, или первый квартиль, столбца.



75 % - семьдесят пятый процентиль, или третий квартиль, столбца.



Контексты и выражения

Это основные компоненты уникального синтаксиса преобразования данных в Polars. Выражения относятся к вычислениям или преобразованиям, которые выполняются над столбцами данных, и позволяют применять различные операции над данными для получения новых результатов.

Контексты в Polars



Выбор



Фильтрация



Группировка/агрегация

Индексирование

```
df[0,:]
```

```
df[:,['fruit', 'price']]
```

Фильтрация строк

```
df.filter(pl.col("price") > 0.5).head()
```

Метод select

```
df.select(['fruit', 'price'])
```

Метод with_columns

```
df.with_columns([  
    (pl.col("price") * pl.col("quantity")).alias("total")  
])
```


Экспресс-опрос

?

Что делает with_columns?



`with_columns`

Это основное выражение для трансформации данных, которое позволяет создать новый столбец для анализа.

Пример

```
df.with_columns([  
    (pl.col("price") * pl.col("quantity")).alias("total")  
])
```



`.with_columns()`

Это метод, который добавляет новые столбцы в DataFrame. Он ожидает список кортежей, где каждый кортеж содержит выражение столбца и необязательный псевдоним.



`pl.col("price") * pl.col("quantity")`

Это выражение, которое вычисляет произведение значений в столбцах «цена» и «количество». `pl.col()` - это функция от Polars, которая позволяет вам ссылаться на определенный столбец в DataFrame.



.alias("total")

Задаёт псевдоним для нового столбца. В данном случае новому столбцу присвоен псевдоним «total».



ВОПРОСЫ



Пример

Скачаем данные по [ссылке](#)

```
import polars as pl
data=pl.read_csv('books_data.csv', null_values='NaN')
print(data.head())
print(data.describe())
```


Экспресс-опрос

?

Что делает `pl.read_csv('books_data.csv', null_values='NaN')`?



```
pl.read_csv('books_data.csv', null_values='NaN')
```

Считывает CSV-файл в Polars DataFrame, указывая 'NaN' в качестве представления для нулевых значений



Обработка пропусков

Каждый столбец в DataFrame (или эквивалентно Series) представляет собой массив Arrow или набор массивов Arrow, основанный на формате Apache Arrow. Отсутствующие данные представляются в Arrow и Polars с помощью нулевого значения. Нулевое пропущенное значение применяется для всех типов данных, включая числовые значения.



NotaNuumber или NaN

Polars также допускает значения NotaNuumber или NaN для столбцов с плавающей запятой. Эти значения NaN рассматриваются как тип данных с плавающей точкой, а не как пропущенные данные. Вы можете вручную определить пропущенное значение с помощью функции python None.



Определение недостающих данных

Прежде чем работать с отсутствующими данными, необходимо определить, где в вашем DataFrame имеются отсутствующие значения. Вы можете использовать такие функции, как `is_null()` или `describe()`, чтобы получить сводку отсутствующих значений в каждом столбце.

Удаление пропущенных данных

```
df = df.drop_nulls() # Отбрасываем строки с любыми пропущенными значениями
```

Заполнение пропущенных данных

```
df = df.fill_null(0) # Замена отсутствующих значений на 0
```

Прямое и обратное заполнение

```
df.with_columns(pl.col('has_nulls').forward_fill()) # Прямое заполнение  
недостающих значений в определенном столбце
```


Интерполяция

```
df = df.interpolate() # Линейная интерполяция для недостающих значений в  
определенном столбце
```

Пример

```
data = data.with_columns(pl.col('year').forward_fill())  
data.describe()
```

Экспресс-опрос



Что такое кастинг?



Кастинг

Преобразует базовый DataType столбца в новый. Polars использует Arrow для управления данными в памяти и полагается на вычислительные ядра в реализации Rust для выполнения преобразования. Доступен с помощью метода `cast()`

Экспресс-опрос



Что такое cast?



cast

Это метод, который включает параметр `strict`, определяющий поведение Polars при встрече со значением, которое не может быть преобразовано из исходного `DataType` в целевой `DataType`.

Пример

```
data = data.with_columns(
    data['year'].cast(pl.Int32) # or pl.Int64 if larger values are expected
)
data.head(5)
```

Экспресс-опрос

?

Что такое quantile()?



quantile()

Вычисляет значения квантилей для числовых данных в DataFrame.

Экспресс-опрос

?

Что такое mean()?



mean()

Вычисляет среднее (усредненное) значение числовых данных в каждой группе. Это мера центральной тенденции, которая позволяет понять типичное значение в каждой группе.



`sum()`

Вычисляет сумму числовых данных в каждой группе.



`min()` и `max()`

Вычисляют минимальное и максимальное значения соответственно в каждой группе.



count()

Подсчитывает количество ненулевых значений в каждой группе.

Экспресс-опрос

?

Что такое `std()` и `var()`?



`std()` и `var()`

Вычисляют стандартное отклонение и дисперсию, соответственно, в каждой группе.



Медиана

Это среднее значение в отсортированном списке числовых данных, устойчивое к промахам.

Столбец describe()

```
df.select([
    pl.col("price").count().alias("count"),
    pl.col("price").null_count().alias("null_count"),
    pl.col("price").mean().alias("mean"),
    pl.col("price").std().alias("std_dev"),
    pl.col("price").median().alias("median"),
    pl.col("price").min().alias("min"),
    pl.col("price").quantile(0.25).alias("25%"),
    pl.col("price").quantile(0.5).alias("50%"),
    pl.col("price").quantile(0.75).alias("75%"),
    pl.col("price").max().alias("max"),
])
```



```
pl.col("price").count().alias("count")
```

Эта операция подсчитывает количество ненулевых значений в столбце "price" и присваивает результат новому столбцу с именем "count".



```
pl.col("price").null_count().alias("null_count")
```

Эта операция вычисляет количество нулевых (отсутствующих) значений в столбце "price" и присваивает результат новому столбцу с именем "null_count".



```
pl.col("price").mean().alias(  
"mean")
```

Эта операция вычисляет среднее значение столбца "price" и присваивает результат новому столбцу с именем "mean".



```
pl.col("price").std().alias("std_dev")
```

Эта операция вычисляет стандартное отклонение значений в столбце "price" и присваивает результат новому столбцу с именем "std_dev".



```
pl.col("price").median(  
).alias("median")
```

Эта операция вычисляет медианное значение столбца "price" и присваивает результат новому столбцу с именем "median".



```
pl.col("price").min().alias("min")
```

Эта операция находит минимальное значение в столбце "price" и присваивает результат новому столбцу с именем "min".



```
pl.col("price").quantile(0.25).  
alias("25%")
```

Эта операция вычисляет 25-й процентиль (первый квартиль) значений в столбце "price" и присваивает результат новому столбцу с именем "25%".



```
pl.col("price").quantile(0.5)  
.alias("50%")
```

Эта операция вычисляет 50-й процентиль (медиану) значений в столбце "price" и присваивает результат новому столбцу с именем "50 %".



```
pl.col("price").quantile(  
0.75).alias("75%")
```

Эта операция вычисляет 75-й процентиль (третий квартиль) значений в столбце "price" и присваивает результат новому столбцу с именем "75 %".



```
pl.col("price").max().alias("max")
```

Эта операция находит максимальное значение в столбце "price" и присваивает результат новому столбцу с именем "max".

Экспресс-опрос

?

Что такое groupby?



groupby

Это операция для группировки данных на основе одного или нескольких столбцов и выполнения агрегированных операций над этими группами



groupby()

Это функция, которая используется для группировки данных по одному или нескольким столбцам, создавая группы на основе уникальных значений в этих столбцах.

Синтаксис

```
DataFrame.groupby([col1, col2, ...])
```


Экспресс-опрос

?

Что такое agg()?



agg()

Это функция, которая используется для указания операций агрегирования, которые должны быть выполнены над каждой группой. Она принимает словарь, в котором ключами являются имена столбцов, а значениями - применяемые функции агрегирования.

Синтаксис

```
GroupBy.agg({'col1': 'agg_func1', 'col2': 'agg_func2', ...})
```

Экспресс-опрос

?

Что такое first() и last()?



first() и last()

Возвращает первое или последнее значение в каждой группе.

Экспресс-опрос

?

Что такое pivot()?



`pivot()`

Эта функция изменяет форму данных на основе значений указанных столбцов, создавая разворотную таблицу. Применяется для создания перекрестных таблиц и обобщения данных.

Пример

```
data.groupby("type").agg(
    [
        pl.mean("price").alias("mean_price"),
        pl.median("year").alias("median_year"),
        pl.count(),
    ]
)
```


Пример

```
data.groupby(["type", "year"]).agg([
    pl.count()
]).pivot(
    values="count",
    index=["year"],
    columns=['type'])
```

Экспресс-опрос

?

Что такое оконные функции?



Оконные функции

Это функции, которые позволяют выполнять вычисления в определенном окне строк в DataFrame, обеспечивают гибкий способ вычисления агрегатов, ранжирования и других операций в определенном окне, таких как скользящее среднее или кумулятивная сумма.

Экспресс-опрос

?

Что такое over()?



over()

Это функция, которая используется для указания окна, над которым работает оконная функция. Она определяет границы окна на основе критериев разбиения и упорядочивания.

Синтаксис

```
over([partition_by, order_by, frame])
```



`sum()`

Вычисляет сумму значений в пределах окна.



mean()

Вычисляет среднее значение (average) значений в окне.



`min()` и `max()`

Вычисляет минимальное и максимальное значения соответственно в пределах окна.



count()

Подсчитывает количество строк в окне.

Экспресс-опрос

?

Что такое rank()?



rank()

Это функция, которая присваивает ранг каждой строке в окне и последовательные ранги.

Экспресс-опрос

?

Что такое rolling()?



rolling()

Это удобный способ создания скользящего окна для данных временных рядов. Она позволяет задать размер окна и применить агрегатные функции в пределах этого окна.

Синтаксис

```
rolling(window_size).agg([col1, col2, ...])
```

Экспресс-опрос

?

Что такое shift()?



shift()

Это функция, которая позволяет сдвинуть значения определенного столбца на заданное количество строк вперед или назад.

Синтаксис

```
shift(offset)
```

Пример

```
data.with_columns([  
    pl.col("price").mean().over("type").alias("mean_price_by_type")  
]).head(10)
```

Способы подсчета уникальных значений

Точная методика

Аппроксимация

Пример

```
data.select(
  pl.col("year").n_unique().alias("unique"),
  pl.approx_n_unique("year").alias("unique_approx"),
)
```



Сложные фильтры

Позволяют применять расширенные условия фильтрации к фрейму данных, позволяя выбирать определенные строки на основе сложных критериев. Могут включать в себя несколько условий, логические операторы и даже пользовательские функции.



Условия фильтрации

Это выражения, которые оцениваются как True или False для каждого ряда в DataFrame. Они могут включать в себя сравнения, логические операторы и функции.



Логические операторы

& (AND), | (OR) и ~ (NOT) - операторы, которые можно использовать для объединения нескольких условий.



Операторы сравнения

`==` (равно), `!=` (не равно), `<` (меньше), `>` (больше), `<=` (меньше или равно) и `>=` (больше или равно), используются для сравнения значений.

Пример

```
df.filter((pl.col("возраст") > 30) & (pl.col("пол") == "Мужской"))
```

```
df.filter(~(pl.col("category") == "High"))
```



Функции

Для выполнения сложных вычислений или преобразований можно использовать встроенные функции или функции, определяемые пользователем в условиях фильтрации.

Пример

```
def is_adult(age):  
    return age >= 18  
  
df.filter(is_adult(pl.col("age")))
```



Цепочка фильтров

Несколько условий фильтрации можно объединить в цепочку, чтобы создать более сложные критерии фильтрации.

Синтаксис

```
DataFrame.filter(condition1).filter(condition2)
```



Комбинирование фильтров

Фильтры можно комбинировать с другими операциями DataFrame, такими как `select()`, `groupby()` и `agg()`, для выполнения сложных манипуляций с данными.



Условия

Условия типа if-else в выражениях с синтаксисом when, then, otherwise. Предикат помещается в выражение when, и когда он оценивается как true, применяется выражение then, в противном случае применяется выражение otherwise (по порядку).

Пример

```
data.select(  
  pl.col("year"),  
  pl.when(pl.col("year") > 2014)  
    .then(pl.lit(True))  
    .otherwise(pl.lit(False))  
    .alias("conditional"),  
)
```



ВОПРОСЫ



Пример



Создадим примеры датасетов для демонстрации операций соединения

Пример

```
import polars as pl
import numpy as np

# Create a random seed for reproducibility
np.random.seed(42)

# Dataset 1: User information
data1 = {
    'user_id': np.arange(1, 101),
    'name': np.random.choice(['Alice', 'Bob', 'Charlie', 'David'], 100)
}
df1 = pl.DataFrame(data1)

# Dataset 2: User age information
data2 = {
    'user_id': np.arange(51, 151),
    'age': np.random.randint(18, 65, size=100)
}
df2 = pl.DataFrame(data2)

# Dataset 3: User scores
data3 = {
    'user_id': np.random.randint(1, 101, size=100),
    'score': np.random.randint(200, 1000, size=100)
}
df3 = pl.DataFrame(data3)
```

Экспресс-опрос

?

Какие Типы объединений доступны в Polars?

Типы объединений, доступные в Polars

Inner Join

Left Join

Outer Join

Cross Join

Semi Join

Anti Join

Inner Join



Описание

Возвращает строки, которые имеют совпадающие значения в обеих таблицах. Используется, когда нужно получить только те записи, которые имеют соответствующие записи в обоих объединяемых DataFrames.

Пример

```
inner_joined_df = df1.join(df2, on='user_id', how='inner')
print("Inner Join Result:\n", inner_joined_df.head())
```

Left Join



Описание

Возвращает все строки из левой таблицы и совпадающие строки из правой таблицы. Если совпадений нет, результатом будет `NULL` на стороне правой таблицы.

Пример

```
left_joined_df = df1.join(df2, on='user_id', how='left')
print("Left Join Result:\n",
      left_joined_df.sample(10).head())
```


Outer Join



Описание

Возвращает все строки из левой и правой таблиц. Если совпадений нет, результатом будет `NULL` на стороне таблицы без совпадений. Заполняет недостающие значения в столбцах соединения.

Пример

```
full_outer_joined_df = df1.join(df2, on='user_id',
                                how='outer')
print("Full Outer Join Result:\n",
      full_outer_joined_df.sample(10).head())
```

Cross Join



Описание

Получает декартово произведение двух таблиц. Каждая строка первой таблицы объединяется со всеми строками второй таблицы

Пример

```
df_brands = pl.DataFrame({
    'brand': ['lacoste', 'prada', 'calvin klein']
})
df_sizes = pl.DataFrame({
    'sizes': ['S', 'M', 'L']
})
cross_joined_df = df_brands.join(df_sizes, how='cross')
print("Cross Join Result (first 5 rows):\n",
      cross_joined_df)
```

Semi Join



Описание

Возвращает все строки из левой таблицы, для которых есть совпадающие ключи в правой таблице, но не включает ни одного столбца из правой таблицы.

Пример

```
semi_joined_df = df1.join(df2, on='user_id', how='semi')
print(f"Semi Join Result {semi_joined_df.shape}:\n",
      semi_joined_df.head())
```

Anti Join



Описание

Возвращает все строки из левой таблицы, для которых нет совпадающих ключей в правой таблице.

Пример

```
anti_joined_df = df1.join(df2, on='user_id', how='anti')
print(f"Anti Join Result {anti_joined_df.shape}:\n",
      anti_joined_df.head())
```

Экспресс-опрос

?

В чем особенность соединения asof?



Соединение asof

Это специализированный тип операции соединения, предназначенный в первую очередь для данных временных рядов, где целью является объединение двух наборов данных на основе ближайшего ключа, а не точного совпадения.

Основные характеристики Asof Join



Ближайшее совпадение ключей



Направленность



Требование сортировки



Обработка несовпадающих ключей



backward



forward



nearest



Пример с использованием Polars

```
import polars as pl

df_q = pl.DataFrame({
    "time": ["2024-01-01 10:00", "2024-01-01 10:02", "2024-01-01 10:05"],
    "quantity": [10, 20, 30]
}).with_columns(pl.col("time").str.strptime(pl.Datetime)).sort('time')

df_p = pl.DataFrame({
    "time": ["2024-01-01 10:01", "2024-01-01 10:03", "2024-01-01 10:06"],
    "price": [100, 200, 300]
}).with_columns(pl.col("time").str.strptime(pl.Datetime)).sort('time')

# Выполнение объединения по принципу asof
result = df_q.join_asof(df_p, on="time")
print(result)
```


Экспресс-опрос

?

Что такое tolerance?



tolerance

Это важная функция, которая позволяет указать предел расстояния между ключами объединения, чтобы строки считались совпадающими.

Параметр допуска



В Polars может быть задан с помощью объекта `timedelta` при работе с типами данных, похожими на время. Это позволяет задавать допуски в единицах времени.

Пример использования tolerance

```
from datetime import timedelta

# Выполняем объединение asof с допуском 1 минута
result = df_q .join_asof(df_p, on=«time», tolerance=timedelta(minutes=1))
print(result)
```

Экспресс-опрос

?

Что такое Merge_sorted?



Merge_sorted

Это специализированный метод, предназначенный для объединения двух уже отсортированных DataFrame на основе указанного ключа, гарантируя, что результирующий DataFrame останется отсортированным.

Ключевые особенности merge_sorted



Сохранение порядка

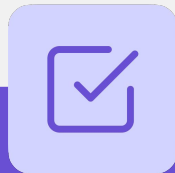


Требование к предварительно отсортированным рамкам данных



Совпадение схем

Основной синтаксис функции merge_sorted



```
DataFrame.merge_sorted(other, key)
```

- other: Другой DataFrame, который должен быть объединен с текущим DataFrame
- ключ: Имя столбца или список имен столбцов, по которым отсортированы DataFrame и которые будут использоваться в качестве ключа слияния

Пример использования merge_sorted

```
df11 = pl.DataFrame({
    "Name": ["John", "Joseph", "Albert"],
    "Age": [18, 15, 29]
}).sort("Age")

df12 = pl.DataFrame({
    "Name": ["Ema", "Andrew", "Michel"],
    "Age": [22, 30, 16]
}).sort("Age")

df13 = df11.merge_sorted(df12, "Age")
print(df13)
```

Экспресс-опрос



Что такое конкатенация?



Конкатенация

Это операция, которая используется для объединения DataFrames или Series как по вертикали (укладывая их друг на друга), так и по горизонтали (растягивая их по сторонам).



Вертикальная конкатенация

Это операция, которая объединяет фреймы DataFrames, складывая их друг на друга.

Вертикальная конкатенация

```
df21 = pl.DataFrame({
    "Name": ["Alice", "Bob"],
    "Age": [25, 30]
})

df22 = pl.DataFrame({
    "Name": ["Charlie", "David"],
    "Age": [35, 40]
})

df_vertical = pl.concat([df21, df22])
print(df_vertical)
```

Пример

shape: (4, 2)

Name	Age
---	---
str	i64
Alice	25
Bob	30
Charlie	35
David	40

Вывод



Горизонтальная конкатенация

Это операция, которая объединяет фреймы
DataFrames, растягивая их рядом друг с другом.

Горизонтальная конкатенация

```
df23 = pl.DataFrame({
    "City": ["New York", "Los Angeles"],
    "Occupation": ["Engineer", "Doctor"]
})

df_horizontal = pl.concat([df21, df23],
    how="horizontal")
print(df_horizontal)
```

Пример

shape: (2, 4)

Name	Age	City	Occupation
---	---	---	---
str	i64	str	str
Alice	25	New York	Engineer
Bob	30	Los Angeles	Doctor

Вывод



Диагональная конкатенация

Это мощный метод объединения DataFrames, которые могут иметь разные схемы. Эта операция не только добавляет записи из нескольких DataFrames, но и управляет различными столбцами, вводя нулевые значения там, где данные отсутствуют в исходных DataFrames.

Диагональная конкатенация

```
df_diagonal = pl.concat(
    [
        df21,
        df23,
    ],
    how="diagonal",
)
print(df_diagonal)
```

Пример

shape: (4, 4)

Name	Age	City	Occupation
---	---	---	---
str	i64	str	str
Alice	25	null	null
Bob	30	null	null
null	null	New York	Engineer
null	null	Los Angeles	Doctor

Вывод

Конкатенация

Имена столбцов

Типы данных

Для вертикальной конкатенации имена столбцов в DataFrames должны совпадать. Если они не совпадают, возможно, потребуется переименовать столбцы перед конкатенацией.

Конкатенация

Имена столбцов

Типы данных

Типы данных конкатенируемых столбцов должны быть совместимы. При несовпадении Polars может выполнить неявное приравнивание типов, что может привести к потере данных или ошибкам.

Rechunking



Если вы не хотите, чтобы Polars повторно копировал конкатенированный DataFrame, укажите `rechunk = False` при выполнении конкатенации

Экспресс-опрос



Что такое Melt?



Melt

Это функция, используемая для преобразования DataFrame из широкого формата в длинный. Разворачивает DataFrame, превращая столбцы в строки.

Переменные-идентификаторы (id_vars)

Это столбцы в DataFrame, которые используются для идентификации каждой строки.



Важно:

Переменные-идентификаторы остаются в вертикальном формате и при необходимости дублируются в результирующем длинном формате DataFrame.

Переменные значения (value_vars)

Эти столбцы не имеют поворотов и преобразуются в два столбца в длинном формате - один для имен переменных и один для значений.



Важно:

Если value_vars не указан, все столбцы, не заданные как id_vars, используются как переменные значения.

Имя переменной и имя значения

Можно задать имена для новых столбцов, в которых будут храниться имена переменных и их значения.



Важно:

Если они не указаны, используются значения по умолчанию (обычно «variable» для имен неповоротных столбцов и «value» для соответствующих значений).

Пример использования Melt

```
df = pl.DataFrame({
    "Year": [2020, 2021],
    "Product A": [100, 150],
    "Product B": [90, 120]
})

melted_df = df.melt(id_vars="Year",
    value_vars=["Product A", "Product B"],
    variable_name="Product",
    value_name="Sales")

print(melted_df)
```

Пример

shape: (4, 3)

Year	Product	Sales
---	---	---
i64	str	i64
2020	Product A	100
2021	Product A	150
2020	Product B	90
2021	Product B	120

Вывод

Melt

Целостность данных

Производительность

Обработка пропущенных
данных

Переменные-идентификаторы (`id_vars`) однозначно идентифицируют каждую строку в исходных данных, чтобы избежать проблем с дублированием данных в новом DataFrame.

Melt

Целостность данных

Производительность

Обработка пропущенных
данных

Для повышения
производительности
рекомендуется перед melt
отфильтровать или
уменьшить размер
DataFrame, если это
возможно.

Melt

Целостность данных

Производительность

Обработка пропущенных
данных

В зависимости от ваших
данных и контекста
операции, вам может
понадобиться заполнить или
обработать нули после
преобразования.



ВОПРОСЫ



Экспресс-опрос



Что такое ленивые вычисления?



Ленивые вычисления

Это вычислительная стратегия, которая откладывает вычисление выражений до тех пор, пока не понадобятся их значения.

Преимущества ленивых вычислений



Оптимизированная производительность



Эффективность использования памяти



Гибкость в обработке данных



Улучшенная обработка ошибок

Ленивые вычисления



Polars использует ленивые вычисления для анализа и оптимизации всего конвейера, гарантируя, что данные будут обработаны наиболее эффективным способом



LazyFrame

Это структура данных в Polars, которая представляет собой отложенные вычисления или серию преобразований, применяемых к набору данных.

От DataFrames к LazyFrames

```
import polars as pl

df = pl.DataFrame({
    "name": ["Alice", "Bob", "Charlie"],
    "age": [25, 30, 35],
    "city": ["New York", "Los Angeles", "Chicago"]
})

lazy_df = df.lazy()
```

Чтение данных в LazyFrames

```
lazy_df_from_csv = pl.read_csv("events.csv").lazy()
```

Создание LazyFrames с нуля

```
lazy_df_from_scratch = pl.DataFrame({  
    "id": [1, 2, 3],  
    "value": [10, 20, 30]  
}).lazy()
```

Операции scan_

```
lazy_df_from_csv_scan = pl.scan_csv("very_large_dataset.csv")
```

```
lazy_df_from_parquet_scan = pl.scan_parquet("very_large_dataset.parquet")
```

Операции scan_

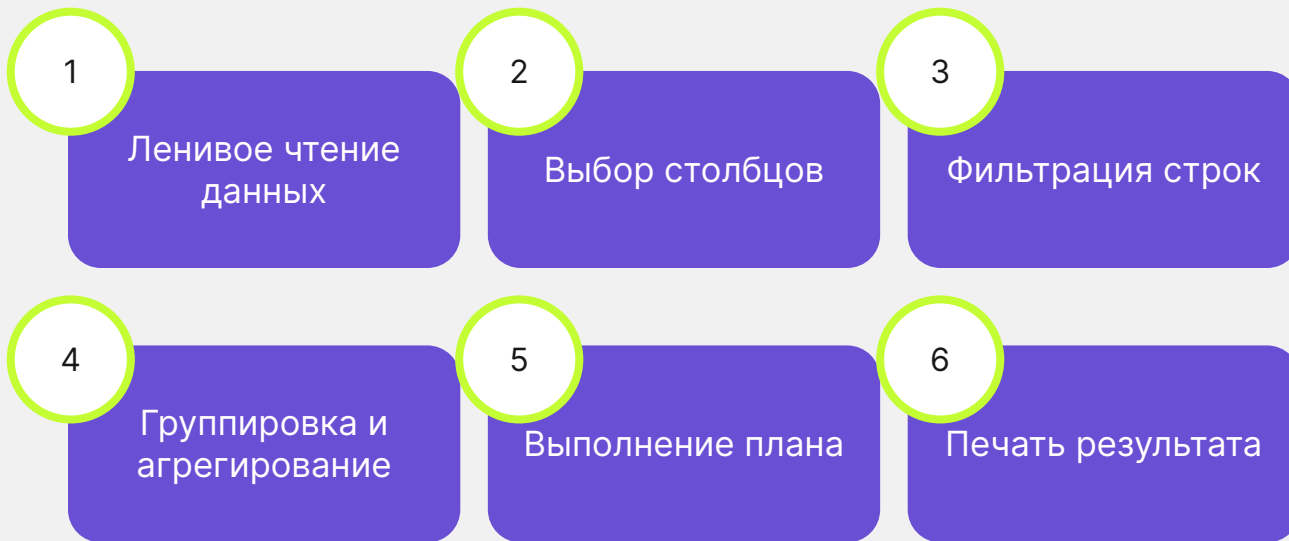
```
result_df = lazy_df_from_csv_scan.collect()
```


Датасет

```
result_df = (
    pl.scan_csv("events.csv")
    .select(['brand', 'event_type', 'price'])
    .filter(pl.col('event_type') == 'view')
    .group_by("brand").agg([pl.col("price").mean().alias("average_price")])
).collect()

print(result_df)
```

Разбор кода





План выполнения

Это представление последовательности операций, которые планируется выполнить над набором данных. При использовании ленивых вычислений эти операции не выполняются сразу, а собираются в вычислительный граф.

Экспресс-опрос

?

Что такое вычислительный граф?



Вычислительный граф

Описывает зависимости между операциями и порядок их выполнения. Визуализация графа может дать ценные сведения об эффективности и структуре конвейера обработки данных.

Зачем визуализировать планы выполнения

Возможности оптимизации

Отладка

Обучение и понимание

Визуализация планов выполнения



Для использования методов визуализации следует установить [Graphviz](#) и добавить путь к бинарным файлам (\bin) в системную переменную PATH.



`.show_graph()`

Это метод, который можно вызвать на LazyFrame для создания визуального представления плана выполнения. Этот метод выводит график, который показывает последовательность операций и их взаимосвязь.

Экспресс-опрос



Что такое collect?



collect

Это метод, который является важнейшим компонентом работы с объектами LazyFrame, особенно при работе с большими наборами данных или потоковыми данными. Используется для выполнения запланированных операций над LazyFrame и сбора результатов в ожидающий вычислений DataFrame.

Экспресс-опрос

?

Что такое streaming?



streaming

Это параметр в методе `collect`, который позволяет Polars обрабатывать и возвращать данные в потоковом режиме. При значении `True` метод `collect` обрабатывает данные по частям, а не загружает в память сразу весь набор данных.

Потоковый параметр

```
import polars as pl

lazy_df = pl.scan_csv("events.csv").filter(pl.col("price") > 100)

result_df = lazy_df.collect(streaming=True)
```

Потоковая обработка

Обработка чанков

Данные обрабатываются кусками.
В зависимости от операций и
порядка их применения это может
повлиять на конечные результаты,
особенно для операций,
требующих глобального
представления данных

Компромиссы с
производительностью

Потоковая обработка

Обработка чанков

Компромиссы с
производительностью

Необходимо учитывать влияние на
производительность и
сопоставлять его с
преимуществами эффективности
использования памяти.

Сравнение скорости выполнения

```
%%timeit

import pandas as pd

df = pd.read_csv("events.csv")

df_selected = df[['brand', 'event_type',
'price']]

df_filtered =
df_selected[df_selected['event_type'] == 'view']

result_df =
df_filtered.groupby('brand')['price'].mean().reset_index(name='average_price')

print(result_df)
```

Pandas: время выполнения: 3.82 s ± 59.7 ms per loop
(mean ± std. dev. of 7 runs, 1 loop each)

```
%%timeit

import polars as pl

result_df = (
    pl.scan_csv("events.csv")
    .select(['brand', 'event_type', 'price'])
    .filter(pl.col('event_type') == 'view')

    .group_by("brand").agg([pl.col("price").mean().alias("average_price")])
    .collect()

print(result_df)
```

Polars: время выполнения: 222 ms ± 15.2 ms per loop
(mean ± std. dev. of 7 runs, 1 loop each)

Цикл в жадном режиме

```
df_list = []
for file in file_list:
    df_list.append(
        pl.read_csv(file)
    )

df = pl.concat(df_list)

output = (
    df
    .groupby('brand')
    .agg([pl.col("price").mean().alias("average_price")])
)
```

Цикл в ленивом режиме

```
df_list = []
for file in file_list:
    df_list.append(
        pl.scan_csv(file)
    )

df = pl.concat(df_list)

output = (
    df
    .groupby('brand')
    .agg([pl.col("price").mean().alias("average_price")])
).collect()
```

Упрощение

```
output = (  
    pl.scan_csv("./*.csv")  
    .groupby('brand')  
    .agg([pl.col("price").mean().alias("average_price")])  
).collect(streaming = True)
```



ВОПРОСЫ





ЗАДАНИЕ





Задание

1. Загрузите данные на свой жёсткий диск.
2. Считайте их в polars.
3. Определите тип данных каждого столбца.
4. Поменяйте типы данных на необходимые (замените строку на дату).
5. Постройте таблицу описательных статистик.
6. Посчитайте уникальное количество брендов двумя разными способами и покажите их разницу.
7. Замените отсутствующие бренды значением по выбору и убедитесь, что описательная статистика поменялась.

8. Возьмите случайные 5000 строк из датасета и постройте таблицу:

brand	mean_price	max_price	min_price
str	f64	f64	f64
"berkut"	129.706923	256.22	87.7
"orico"	140.315	271.46	9.17
"toshiba"	113.563333	317.75	45.84
"alligator"	34.37	34.37	34.37
"cameronsino"	23.908947	42.06	9.0

9. Постройте pivot таблицу следующего вида:

event_type	palmexx	msi	unknown	xp-pen	thermaltake	canon	bradex
str	f64	f64	f64	f64	f64	f64	f64
"view"	28.238462	399.441565	81.673358	97.78	102.731538	165.018916	21.73
"purchase"	16.98	432.653636	54.197015	null	null	94.622857	null
"cart"	12.22	274.745	72.529219	97.78	152.08	154.33	null

10. На основе оконных функций постройте поле, состоящее из средней цены в разбивке по бренду.



Задание 1

Создайте DataFrame с 1 000 строк и следующими столбцами:

1. id: Последовательные целые числа от 1 до 1 000.
2. имя: Случайные имена из предопределенного списка.
3. возраст: Случайный возраст от 18 до 65 лет.
4. salary: Случайная зарплата в диапазоне от 30 000 до 100 000.
5. отдел: Случайные названия отделов из предопределенного списка.
6. join_date: Случайные даты в течение последних 10 лет.

Задание 2

Базовые преобразования:

1. Фильтрация: Отфильтровать сотрудников, которые относятся к отделу "ИТ" и старше 30 лет.
2. Сортировка: Отсортируйте отфильтрованный DataFrame по зарплате в порядке убывания.
3. Выбор столбцов: Выберите только id, имя и зарплату из отсортированных DataFrame.

Задание 3

Расширенные преобразования:

1. Группировка по и агрегирование: Вычислите среднюю зарплату по отделу.
2. Melt: Преобразуйте DataFrame из широкого формата в длинный, используя id и name в качестве переменных-идентификаторов.
3. Join: Создайте небольшой DataFrame с названиями отделов и их местоположением. Выполните внутреннее объединение с исходным DataFrame, чтобы добавить информацию о местоположении.

Задание 4



Использование ленивых вычислений:

1. Преобразуйте исходный DataFrame в LazyFrame.
2. Применить преобразования: Примените ряд преобразований с помощью ленивого API (фильтр, выборка, группировка по и агрегирование).
3. Соберите результат в DataFrame.

Задание 5

Визуализация:

1. Гистограмма: Создайте гистограмму распределения по возрасту.
2. Гистограмма: Создайте гистограмму, показывающую среднюю зарплату по отделам.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Загрузите данные на свой жёсткий диск.
2. Считайте их в polars.
3. Посчитайте уникальное количество кодов категорий двумя разными способами.
4. Замените отсутствующие коды категорий значением по выбору и убедитесь, что описательная статистика поменялась.
5. На основе оконных функций постройте поле, состоящее из максимальной цены в разбивке по коду категории.
6. Постройте новое поле, принимающее значение True, если цена больше средней цены, и False в ином случае.

Домашнее задание

Используйте набор данных "tips" и набор данных "flights" из Seaborn.

Инструкции:

1. Загрузите наборы данных "tips" и "flights" в отдельные Polars DataFrames.
2. Из набора данных 'flights' выберите столбцы 'year', 'month' и 'passengers'.
3. Создайте новый DataFrame из 'flights', который агрегирует общее количество пассажиров за год.
4. Объедините эти агрегированные данные с набором данных 'tips' по столбцу 'year' (примечание: для этого вам может понадобиться создать столбец 'year' в 'tips').
5. Конкатенируйте DataFrame 'tips' (отфильтрованный по времени 'Dinner') с первыми 50 строками 'flights' (подумайте, каким образом).
6. Отобразите первые 10 строк объединенного и конкатенированного DataFrame.

Решение

```
import seaborn as sns
import polars as pl

tips_df = pl.from_pandas(sns.load_dataset('tips'))
flights_df = pl.from_pandas(sns.load_dataset('flights'))

flights_agg = flights_df.select(['year', 'passengers']) \
    .group_by('year') \
    .agg(pl.col('passengers').sum().alias('total_passengers'))

tips_df = tips_df.with_columns(pl.lit(2016).cast(pl.Int64).alias('year'))

joined_df = tips_df.join(flights_agg, on='year', how='left')

tips_dinner = tips_df.filter(pl.col('time') == 'Dinner')

concatenated_df = pl.concat([tips_dinner, flights_df.head(50)], how='diagonal')

print("Joined DataFrame:")
print(joined_df.head(10))

print("Concatenated DataFrame:")
print(concatenated_df.head(10))
```



ВОПРОСЫ



Заключение

